

Internet stvari i servisa

Projekat 2

IoT mikroservisi zasnovani na događajima – uporedna evaluacija MQTT-a i Kafke

1. Cilj projekta

Istražiti performanse, skalabilnost i ograničenja različitih message broker sistema zasnovanih na publish-subscribe modelu u IoT mikroservisnim arhitekturama. Fokus je na razumevanju *trade-off* odluka (kašnjenje vs. pouzdanost) i pogodnosti ovih sistema za *edge* i *cloud* okruženja.

Za zadatak se koristi isti IoT dataset i model podataka iz prvog projekta (uz mogućnost proširenja atributa). Kompletan sistem mora biti kontejnerizovan pomoću **Docker Compose-a**.

Mogu se koristiti najmanje dve tehnologije (ASP.NET Core, Node.js, Spring Boot, FastAPI itd.).

2. Mikroservisna arhitektura:

Projektovati asinhroni, *event-driven* mikroservisni sistem koji se sastoji od sledećih komponenti:

- **Data Ingestion Service:** Simulira IoT uređaje i šalje podatke u realnom vremenu na odgovarajući broker (MQTT topic ili Kafka topic).
- **Data Storage Service:** Pretplaćen na broker, preuzima poruke i skladišti ih u PostgreSQL bazu podataka. **Važna napomena za optimizaciju:** Tokom *stress-testova* visokog intenziteta (Scenariji A i C), implementirati **batching** (grupni upis na svakih 500 poruka) ili privremeno isključiti upis u bazu kako I/O podsistem računara ne bi postao usko grlo umesto samog brokera.
- **Analytics Service (Stream Processing):** Pretplaćen na tok podataka i implementira jednostavan Tumbling Window (fiksni vremenski prozor) od 10 sekundi u cilju agregacije, filtriranja, statističke analize, detekciju anomalija ili pragova. Logika: Za svakih 10 sekundi računa prosečnu vrednost senzora (npr. temperature). Ako je prosek prozora veći od definisanog praga (npr. $> 50^{\circ}\text{C}$), servis ispisuje kritičan alarm (Alert) u log.

3. Implementirati istu arhitekturu korišćenjem dva message broker-a

MQTT (Mosquitto)

- Testirati ponašanje sistema menjanjem nivoa kvaliteta usluge: QoS 0, QoS 1 i QoS 2.
- Analizirati efekte garancije isporuke poruka na latenciju (at most once/at least once/exactly once) efekta

Apache Kafka

- Koristiti KRaft režim (bez Zookeeper-a) radi uštede memorijskih resursa na lokalnim mašinama.
- Konfigurisati parametre potvrde prijema: acks=0, acks=1 i acks=all.
- Analizirati pojam Consumer Lag-a i partitionisanja.

4. Eksperimentalni IoT scenariji

- Scenario A (Massive Sensor Ingestion): Simulirati paralelni rad 100, 1000 i 10000 uređaja. Pratiti maksimalni throughput (broj poruka u sekundi) i procenat izgubljenih poruka.
- Scenario B (Edge Connectivity Failures): Pomoću komande `docker network disconnect` simulirati mrežni prekid na simulatoru uređaja u trajanju od 30 sekundi. Pratiti recovery mehanizme oba brokera nakon ponovnog povezivanja (trajne pretplate kod MQTT-a vs. pomeranje offset-a kod Kafke).
- Scenario C (Burst Event Load): Generisati nagli skok sa 50 na 5000 poruka/s u trajanju od nekoliko sekundi. Pratiti formiranje reda čekanja (backlog), backpressure ponašanje i vreme potrebno da se sistem vrati u normalu (recovery time).

- Scenario D (Real-Time Alerting): Izmeriti end-to-end latenciju – vreme od trenutka kada simulator generiše kritičnu vrednost do trenutka kada Analytics Service ispiše alarm.

5. Merenje performansi

Za generisanje opterećenja i prikupljanje metrika obavezno je koristiti namenske alate:

- **Za MQTT testiranje:** Koristiti zvanični **emqtt-bench** alat ili k6 sa MQTT ekstenzijom.
- **Za Kafka testiranje:** Koristiti native skripte visokih performansi koje dolaze uz samu Kafku (**kafka-producer-perf-test.sh**) ili k6 sa xk6-kafka dodatkom.
- **Za praćenje resursa:** Koristiti **docker stats** (opciono: Prometheus + Grafana stack) za beleženje CPU, RAM i mrežnog saobraćaja kontejnera.

6. Analiza pouzdanosti i različitih nivoa garancije isporuke.

U okviru izveštaja, odgovorite na sledeća inženjerska pitanja:

1. Zašto je MQTT idealan za postavljanje na samim *edge* uređajima (senzorima), a zašto postaje neadekvatan kada nam je potrebna istorijska analitika velikih podataka?
2. Zašto Kafka dominira u *data-intensive* cloud sistemima, kolika je "cena" njene skalabilnosti u pogledu resursa i da li je realno pokretati je na hardverski ograničenim *edge* serverima?
3. Popuniti uporednu tabelu performansi (Throughput, p95 latencija, CPU/RAM footprint) na osnovu sprovedenih eksperimenata.

Očekivani rezultati projekta:

1. Git repo
2. Docker Compose konfiguracija
3. Konfiguracija brokera
4. Benchmark skripte
5. Eksperimentalni podaci
6. Tehnički izveštaj sa kratkim opisom urađenog, uporednom tabelom i odgovorima na kritička pitanja